Projet
Projet de jeu vintage

Introduction

Votre tâche est de développer les bases d'un jeu vidéo de type action/aventure ou jeu de rôle comme on en faisait dans les années 1990, en Java, à partir d'une interface graphique qui vous est déjà fournie : `Mazing`.

Après avoir lu ce document, lisez immédiatement la documentation de `Mazing` pour la connaître.

La bibliothèque fournie vous donne tous les éléments graphiques et d'animation du jeu, il faut donc se concentrer sur le reste, c'est-à-dire l'organisation du code, l'application pertinente des *Design Pattern* dans une architecture MVC bien faite.

Il n'est pas autorisé d'utiliser des bibliothèques autres que celles de `java.util`, `java.lang`, `javax.swing` ou `Mazing`.

Le scénario du jeu est à votre discrétion, mais doit être validé par votre responsable de TD. Nous suggérons que le but du jeu est de réussir à trouver la sortie d'un labyrinthe. Mais d'autres objectifs peuvent être définis comme par exemple de récolter un certain nombre de trésors, échapper à des ennemis, ou encore jouer à deux ou plus en concurrence sur un même espace et dans un même temps.

Cet exercice est un exercice pédagogique évalué. Il impose plus une obligation de moyens que de résultat. C'est-à-dire que la livraison d'un jeu de qualité, jouable et amusant sera appréci de l'évaluateur, mais la qualité de l'organisation du code, de l'architecture logicielle et de l'application des concepts vus en cours seront plus appréciés encore.

Il est donc impératif de soigner la qualité de l'organisation de votre projet et de passer beaucoup moins de temps aux aspects graphiques, sonores ou jouables.

Livrables

Chaque projet est réalisé par 3 personnes d'un même groupe de TD. L'une de ces personnes se charge de déposer deux fichiers sur la page Moodle avant le mardi 1er décembre 2026 à midi :

1. Un code source documenté et sourtout propre (clean-code). Ce code est livré dans une archive `.zip` ou `.tgz` contenant tous les fichiers `.java`. Mais en revanche pas les fichiers compilés, ni les bibliothèques. Si vous avez utilisé Gradle, Cmake, Maven, etc. pour compiler le projet, indiquez cela dans le document et livrez les fichiers de configuration pour compiler.
2. Un document `.pdf` contenant
   (a) Une page de garde avec le nom de chaque participant et sa contribution au projet.
   (b) La structure du code (packages, classes, relations entre classes, etc).
   (c) Les Design Patterns utilisés avec une explication des problématiques auxquelles ils répondent.

Évaluation

L'évaluation de ce travail est faite lors des séances de TD en plus de l'évaluation du livrable.

C'est une évaluation individuelle. Chaque membre du projet doit explicitement dire les parties qu'il a réalisé seul et celles qu'il a réalisé en commun avec les autres membres du projet.

La qualité de la conception du programme sera particulièrement appréciée. On pourra juger par exemple de la facilité avec laquelle des ajouts au jeu seront possibles. Le jeu pourra évoluer avec l'ajout d'autres personnages ou objets, l'ajout d'autres actions, l'ajout d'éléments de stratégie ou encore l'interface entre joueurs, etc.

La mise en oeuvre et la pertinence des Design Pattern sera appréciée.

La qualité du code comptera également beaucoup (code lisible, documenté, qui compile).

En revanche, nous répétons que les aspects ergonomiques et graphiques, la qualité du jeu, la jouabilité ou l'intérêt du jeu du point de vue gaming seront moins déterminants, mais nous encourageons ceux et celles qui peuvent ajouter ces avantages à leur projet de le faire dans leur propre intérêt.

Toute absence non justifée pendant les évaluation lors des séances de TD sera notée ABI.

Remarque : tous les éléments qui ont été trouvés dans la littérature ou grâce aux intelligences artificielles devront être identifiés comme tels et avoir un lien vers les références bibliographiques explicites ou vers le site de l'intelligence artificielle. Le contraire est assimilé à de la fraude.

Description du projet

Le jeu se déroule dans un labyrinthe rempli d'objets (au sens vernaculaire autant qu'au sens de la programmation orientée objet) ainsi que de personnages. Les personnages sont capables d'intéragir avec les obsctacles du labyrinthe ainsi qu'avec les autres personnages et les objets. Les éléments du jeu comme des objets et la construction du labyrinthe peuvent avoir des positions prédéfinies au début du jeu, ou bien générées aléatoirement ou encore en interraction avec le joueur. Le jeu doit comporter au moins un personnage ou un élément du jeu controlé par le joueur au clavier ou à la souris.

Le labyrinthe peut être généré automatiquement sur la base d'un graphe dont les sommets représentent les salles. Les sommets sont reliés entre eux si et seulement si il existe une communication (porte, ouverture, etc.) entre eux, c'est-à-dire entre les salles. Les éléments de la théorie des graphes permet de réaliser plusieurs scénarios (plus court chemin entre deux entités du jeu, remplissage d'un labyrinthe par un parcours utilisant une fifo ou une lifo, coloration de graphe pour déterminer deux sommets incompatibles

entre eux, etc.)

Model View Controller

Le projet sera construit autour d'un modèle MVC.

— L'API `Mazing` correspond uniquement à la vue de votre application.

Cependant, il est important d'établir une communication entre la vue et le modèle. Cela permet à la vue de se mettre automatiquement à jour lorsque le modèle change.

— Le modèle doit prendre en charge la totalité de la logique de l'application (les aspects métiers). C'est ici que vous mettrez le labyrinthe et que vous créerez des personnages. C'est également ici que l'on doit traiter les automatismes, comme la dynamique des personnages, leurs états. Si votre jeu évolue dans le temps, c'est encore ici que vous devez traiter cette évolution.

— Mettre en place un contrôleur chargé de transmettre les interactions avec l'utilisateur vers le modèle.

Il est nécessaire de créer une communication entre la vue et le contrôleur. Cela garantit que tous les éléments interactifs de la vue (touches du clavier, `javax.swing.JButton` si vous en mettez, souris si vous l'utilisez, etc.) soient correctement traités par le contrôleur.

L'API `Mazing` fournit une boucle de traitement des animations. C'est également dans le contrôleur, par exemple dans un package `controller.loopAnimations`, que vous devez gérer les déclencheurs et événements liés à cette boucle. Ainsi par exemple, à chaque boucle d'animation d'un personnage, celui-ci peut vérifier sa position, les collisions possibles dans le modèle, etc.

Autres Design Patterns

Il vous est demandé de réaliser au moins 3 autres Design Patterns (création, structure, comportement, ...) que MVC. On considère que le pattern « Observer » fait partie de MVC et ne compte pas dans les 3. Réfléchissez bien aux différents patterns existants et à leur applicabilité au sein de ce projet. Il vous faudra décrire, dans le rapport (en quelques lignes) pourquoi vous avez choisi d'utiliser ces patterns et comment vous les avez mis en oeuvre.